

Informe Inteligencia Artificial 2

Proyecto final

Reconocimiento de órdenes en audio para su integración en un videojuego



Alumno: Tomás Rando

Profesores: Rodrigo Gonzalez y Luciano Robino

Introducción	3
Marco Teórico	3
Espectrogramas Log-Mel	3
CNN	5
Transformers	5
Vision Transformers	6
Diseño Experimental	7
Herramientas	7
Conjuntos de datos	7
Métricas utilizadas	8
Exploración inicial del dataset	8
Implementación del juego	10
Implementación	11
1. CNN simple	11
2. CNN con arquitectura VGG16	12
3. CNN con conexión residual	13
4. Visual Transformers	14
4.1. Visual transformers solo con SpecAugment (v0)	15
4.2. Visual transformers con data augmentation (SpecAugment, ruido, desplazamiento, volumen) (v1)	15
4.3. Visual transformers con data augmentation (SpecAugment, ruido, desplazamiento, volumen, tono) (v2)	15
Resultados	16
Posibles mejoras	22
Conclusiones	22
Bibliografía	24

Introducción

El reconocimiento automático de comandos de voz constituye un área activa dentro del campo del procesamiento de audio. La clasificación de órdenes cortas resulta especialmente relevante en el desarrollo de interfaces entre humanos y sistemas, permitiendo controlar aplicaciones, o en este caso, videojuegos, utilizando instrucciones simples.

El principal problema es lograr modelos capaces de clasificar correctamente palabras pronunciadas por diferentes personas, con ruido de fondo, traslaciones y otras interferencias dadas por el entorno y los dispositivos de audio utilizados. Para encontrar una solución a este problema, se ha utilizado Redes Neuronales Convolucionales y Vision Transformers, que son arquitecturas que tienen un alto rendimiento a la hora de clasificar a partir de representaciones visuales.

En este trabajo se desarrollan modelos con estas arquitecturas y se investiga la capacidad de los mismos para clasificar correctamente órdenes cortas en inglés utilizando sus espectrogramas asociados. Además, para demostrar una aplicación se desarrolla un videojuego simple tipo “ice sliding” que tomando las predicciones de los modelos, ejecute las acciones correspondientes.

En primer lugar, se presenta el marco teórico relacionado al trabajo realizado. Luego, se describe el diseño de experimentos, explicando las evaluaciones realizadas con los modelos, las herramientas utilizadas y los datos de entrada, entre otras cosas. Posteriormente, se presentan los resultados y su correspondiente análisis. Finalmente, se exponen las conclusiones.

Marco Teórico

Espectrogramas Log-Mel

El espectrograma es una herramienta básica de representación. Se utiliza para señales sonoras con su contenido frecuencial. Es una representación en tres dimensiones, temporal, frecuencial y amplitud de la distribución de energía de una señal. Sin embargo, en la práctica suele visualizarse como una imagen bidimensional donde el tiempo y la frecuencia corresponden a los ejes de la imagen, mientras que la amplitud se representa mediante la intensidad del color. Es una representación importante porque la audición humana está basada en un tipo de espectrograma real codificado por la cóclea en el oído interno. [\[3\]](#)

Además, un espectrograma se puede interpretar como la proyección en dos dimensiones de una sucesión de transformadas de Fourier de tramas consecutivas, donde la energía y el contenido frecuencial va variando a lo largo del tiempo. Por ello, para obtener un espectrograma a partir de una señal de audio, se utiliza comúnmente la transformada de Fourier en tiempo corto (STFT). Este método divide la señal en pequeñas ventanas temporales y aplica la transformada de Fourier a cada una de ellas, permitiendo analizar

cómo varía el contenido frecuencial de la señal a lo largo del tiempo. De esta forma, se obtiene información sobre las frecuencias sin perder información temporal.

La elección del tamaño de la ventana de la STFT es importante, pues ventanas más largas permiten obtener mayor información de las frecuencias presentes en la señal, sacrificando precisión temporal sobre los eventos. A su vez, ventanas más chicas ofrecen mejor resolución temporal pero menor precisión a la hora de estimar las frecuencias. Esta información es combinada en un espectrograma, logrando un compromiso entre evolución temporal y contenido frecuencial de la señal.

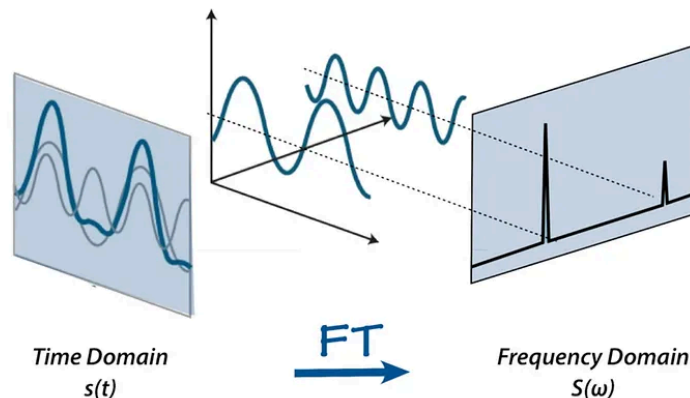


Figura 1. Transformada de Fourier.

En los espectrogramas, cada píxel representa la energía de una determinada frecuencia en un instante de tiempo específico. Para ello, el color de cada uno sigue una escala logarítmica, comúnmente la escala Mel. La misma se utiliza ya que aproxima mejor la forma en la que el oído percibe los sonidos. Un mel es una unidad de pitch definida de manera que pares de sonidos que son perceptualmente equidistantes en un pitch sean separados por el mismo número de mels. La frecuencia mel m puede ser computada a partir de la frecuencia acústica original utilizando la siguiente fórmula:

$$m = 1127 \ln(1 + f / 700)$$

La misma permite que cambios muy pequeños en frecuencias bajas se reflejen casi uno a uno en mels, mientras que cambios en altas frecuencias se compriman y se perciban menos [\[10\]](#).

Esto implica que el eje vertical del espectrograma se vuelve logarítmico, de manera que incrementos uniformes en el eje corresponden aproximadamente a duplicaciones de frecuencia.

Finalmente, el eje horizontal del espectrograma corresponde al tiempo asociado al centro de cada ventana de la STFT. Es decir, el momento en el cual se calcula la transformada de Fourier para ese segmento.

CNN

La CNN es una clase de red neuronal que usa capas convolucionales para filtrar las entradas para quedarse con información útil. Esto se realiza combinando los datos de entrada con un kernel (filtro) para formar un feature map como salida de la capa convolucional. Los filtros o kernels de las capas convolucionales corresponden a los parámetros entrenables del modelo. Esos parámetros se ajustan durante el entrenamiento utilizando backpropagation, permitiendo que la red aprenda a extraer feature maps relevantes para una tarea específica.

Los componentes de una red neuronal convolucional son: una capa de entrada, una capa de salida y una o más capas ocultas. Estas últimas son una combinación de capas convolucionales, capas de pooling, capas de normalización y capas totalmente conectadas.

Las CNN se utilizan principalmente para procesamiento de datos que poseen una estructura espacial, por lo que son especialmente efectivas en tareas relacionadas con imágenes. Algunas de las aplicaciones son: reconocimiento de imágenes, detección de objetos, segmentación de imágenes, reconocimiento de escritura y procesamiento de video, entre otras.

Transformers

Una de las instancias de la arquitectura encoder-decoder es el Transformer. Un transformer básico está compuesto de un encoder y un decoder. En este caso, los embeddings de la entrada se combinan con el encoding posicional antes de ser entregados al encoder y decoder. En la figura 2 es posible observar un esquema del mismo.

El encoder del transformer es una pila de múltiples capas idénticas, donde cada capa posee dos subcapas. La primera de ellas es una capa de self-attention con múltiples cabezales y la segunda es una red feed-forward aplicada por posición. En el caso de self-attention del encoder, las queries, keys y values son las salidas de la capa encoder anterior. Por otro lado, se emplea en ambas capas una conexión residual, seguida por normalización por capas (o layer normalization).

El decoder es también una pila de múltiples capas idénticas con conexiones residuales y normalización por capas. Además, incluye las dos subcapas que poseen los encoders y que fueron descritas en el anterior párrafo. Sin embargo, adicionalmente, el decoder incluye otra subcapa llamada encoder-decoder attention. En ella, las queries vienen desde la salida de la subcapa de auto-atención del decoder, y las keys y values vienen desde las salidas de los encoders. En el caso de la subcapa de auto-atención del decoder, los queries, keys y values vienen dados por las salidas de la capa decoder anterior. Sin embargo, cada posición en el decoder puede atender a todas las posiciones en el decoder hasta ella misma, es decir, las siguientes posiciones se enmascaran para lograr que la predicción sólo dependa de los tokens de salida que fueron generados anteriormente. [\[6\]](#)

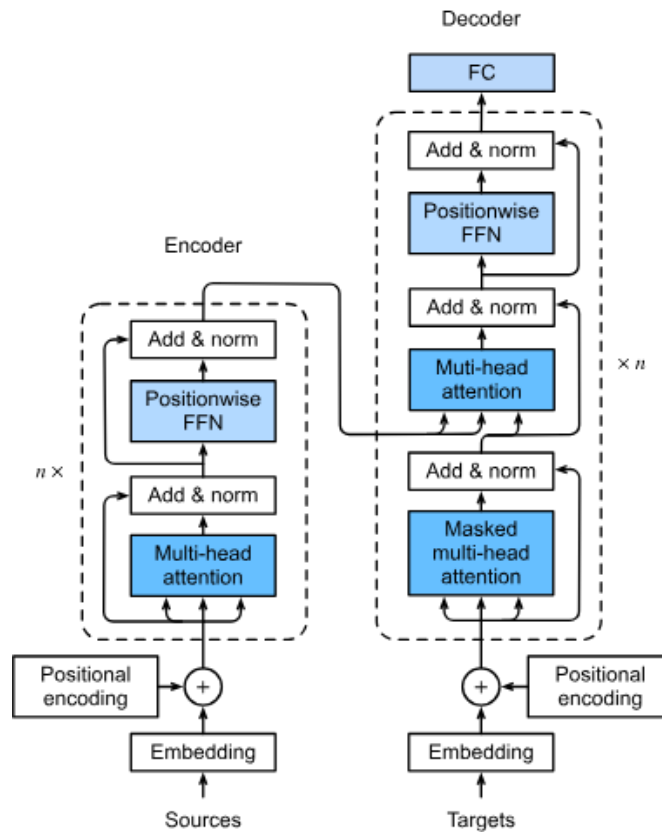


Figura 2. Transformer encoder/decoder.

Existen tres tipos de modos en los que los transformers pueden ser usados: solo encoder (encoder-only), solo decoder (decoder-only), y encoder-decoder. En este trabajo se utilizó un transformer con solo encoder. En este tipo de transformers, una secuencia de tokens de entrada es convertida en el mismo número de representaciones que pueden ser luego proyectadas a la salida (Por ejemplo, para clasificación). Además, los transformers encoder-only consisten en capas de auto-atención donde todos los tokens de entrada se atienden entre sí. [9]

Vision Transformers

Para poder utilizar los transformers con imágenes, los vision transformers cambian la estructura de los datos de entrada. El modelo descompone cada imagen en una secuencia de vectores, imitando la manera en la que los modelos de lenguaje procesan oraciones. Esto se realiza con un cierto proceso.

La imagen se divide en parches, es decir, imágenes más pequeñas de tamaño fijo. Luego, cada parche se aplanan en un vector, convirtiéndose en un token visual. Posteriormente, se pasan por una capa lineal para crear los respectivos embeddings. No obstante, en la mayoría de variantes de ViT se emplean CNN para extraer características y generar los embeddings que se utilizan como entrada del transformer.

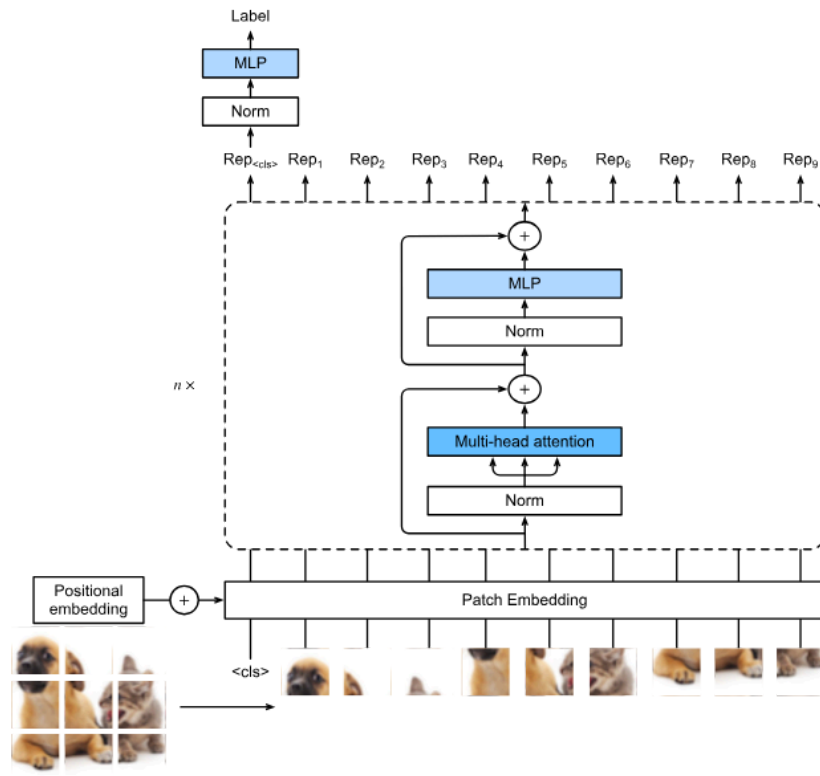


Figura 3. Arquitectura Vision Transformer [\[7\]](#)

Diseño Experimental

Herramientas

Para el desarrollo de los modelos se utilizó Kaggle con Python 3.12.12, TensorFlow 2.19.0, TensorFlow-Datasets 4.9.9, Keras y otras librerías. Por otro lado, para la implementación del juego, se utilizó Godot 4.6.1 [\[8\]](#). Por último, la API fue desarrollada utilizando Python 3.12.12 y Flask 3.1.2.

Conjuntos de datos

Inicialmente, el conjunto de datos se dividió en tres: train, validation y test. El conjunto de train se usó para entrenar los modelos, el de validation para validar el modelo en cada época y el test se utilizó solamente al final para validar el modelo definitivo sin sesgo. Adicionalmente, se creó un cuarto conjunto de tests adicionales con audios propios. Este último con el objetivo de evaluar los modelos con datos más cercanos a los que se utilizarán posteriormente en el juego.

El conjunto de train posee 85511 datos; validation 10102; test 4890; y el de testeo adicional posee 20.

Métricas utilizadas

Para las dos arquitecturas elegidas: CNN y ViT, se elaboraron diferentes arquitecturas y se probaron diferentes configuraciones de estas con el fin de encontrar la mejor mediante las evaluaciones de las mismas. Para ello, se utilizaron diferentes métricas con el objetivo de realizar comparaciones entre las configuraciones. Las mismas fueron:

- Accuracy: El porcentaje de aciertos que el modelo predijo correctamente
- Precision: Mide qué proporción de las predicciones positivas del modelo fueron correctas. En el caso del problema actual, es decir, clasificación multiclase, precision calcula por cada clase: de todas las veces que el modelo predijo la clase, cuántas veces era realmente esa clase, posteriormente, se hace el promedio de los resultados.
- Recall: Mide la proporción de los positivos reales que fueron detectados por el modelo. Nuevamente, en el caso del problema actual, calculamos por cada clase: de todas las veces que realmente era esa clase, cuántas veces el modelo la predijo correctamente, luego, se hace también el promedio de los recalls.
- F1-Score: Combina precision y recall en un solo valor y sirve para medir el equilibrio que existe entre ambas.
- Top-3 Accuracy: Mide si la palabra correcta aparece entre las 3 palabras más probables que predijo el modelo

Estas métricas fueron obtenidas utilizando el conjunto de test. Sin embargo, se utilizó el conjunto de test adicional para calcular un accuracy más por modelo.

Además, para el primer conjunto de datos, se elaboró también la matriz de confusión de cada modelo. Esta matriz nos permite identificar qué palabras fueron las predichas correctamente, y, en el caso de las equivocaciones, permite observar con qué palabras se confundió el modelo.

Por último, una vez los modelos fueron implementados en el juego, se realizaron evaluaciones manuales con el fin de determinar si el modelo predecía correctamente las palabras de entrada al momento de jugar. Esto se realizó con el mejor modelo de cada arquitectura.

Exploración inicial del dataset

Inicialmente, el dataset contenía 100.503 datos, por lo que se dividió el mismo en tres conjuntos: train, validation y test. El de train contiene 85511 datos, el test 4890 y el validation 10102.

Luego de hacer la división, se observó la distribución de los audios según las clases existentes. Ante esto, se pudo identificar un gran desbalance entre las mismas, ya que la clase `_unknown_` poseía 54074 elementos, mientras que el resto tenía aproximadamente unos 3500 ejemplos. Esta situación se abordó de diferentes maneras según la arquitectura utilizada, sin embargo, la solución en general fue hacer undersampling de esta clase, dejándola mejor balanceada con respecto al resto. Se intentó también utilizar pesos con cada clase, pero los resultados de las pruebas no fueron satisfactorios, por lo que se

descartó la idea. La cantidad de elementos por clase por conjunto se muestran en las siguientes figuras.

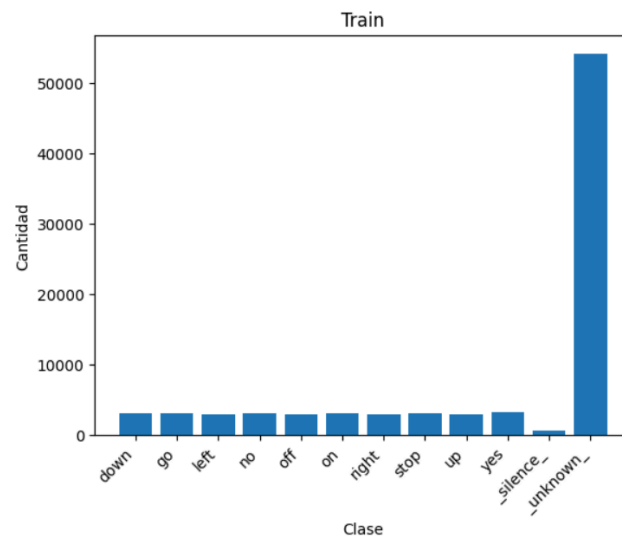


Figura 4. Distribución por clase de los datos del conjunto train.

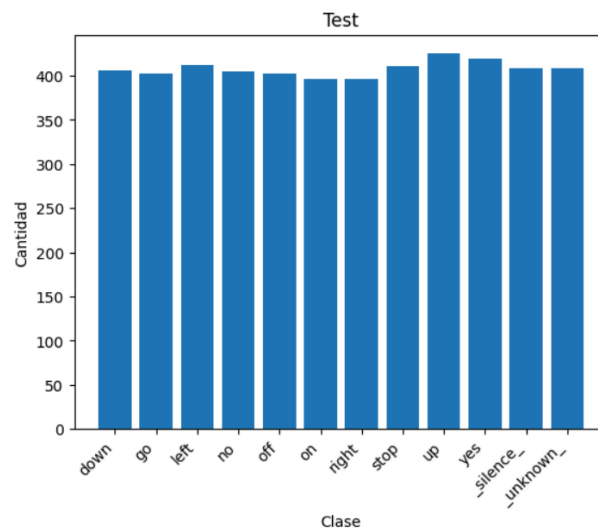


Figura 5. Distribución por clase de los datos del conjunto test.

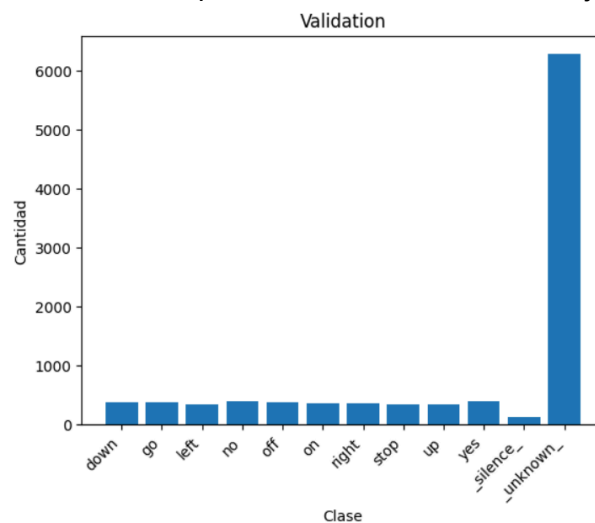


Figura 6. Distribución por clase de los datos del conjunto validation.

Implementación del juego

Según lo definido en la sección introductoria, se desarrolló el videojuego tipo sliding en el que se aplicaron los modelos obtenidos. En el mismo, se incorporaron los mejores modelos de ambas arquitecturas, CNN y ViT. Para el momento de escritura del informe, se implementaron acciones del personaje correspondientes a las palabras “up”; “down”; “left” y “right”. El reconocimiento de dichas palabras provocan que el personaje se mueva en esas direcciones.

El juego actualmente posee un nivel y es posible seleccionar en el menú con qué modelo se desea jugarlo. El personaje inicia en una caverna de hielo en la que hay rocas. El jugador debe mover al personaje utilizando su voz hasta llegar a la escalera. El mecanismo con el que se implementó esto es el siguiente:

- El jugador presiona la barra espaciadora del teclado y el juego comienza a grabar un audio de aproximadamente un segundo y medio.
- El juego guarda el audio y lo envía utilizando el protocolo HTTP hacia una API local.
- La API, que está corriendo los modelos desarrollados, recibe el audio, el modelo lo procesa y devuelve la predicción del mismo.
- El juego recibe la predicción y ejecuta la acción correspondiente.

Este proceso, al estar corriendo ambos componentes localmente, se realiza con una velocidad óptima. De esta manera, el sistema permite una interacción en tiempo real, integrando el reconocimiento de voz a través de espectrogramas y el control del personaje de manera eficiente. Imágenes del videojuego se presentan en las figuras 5 y 6

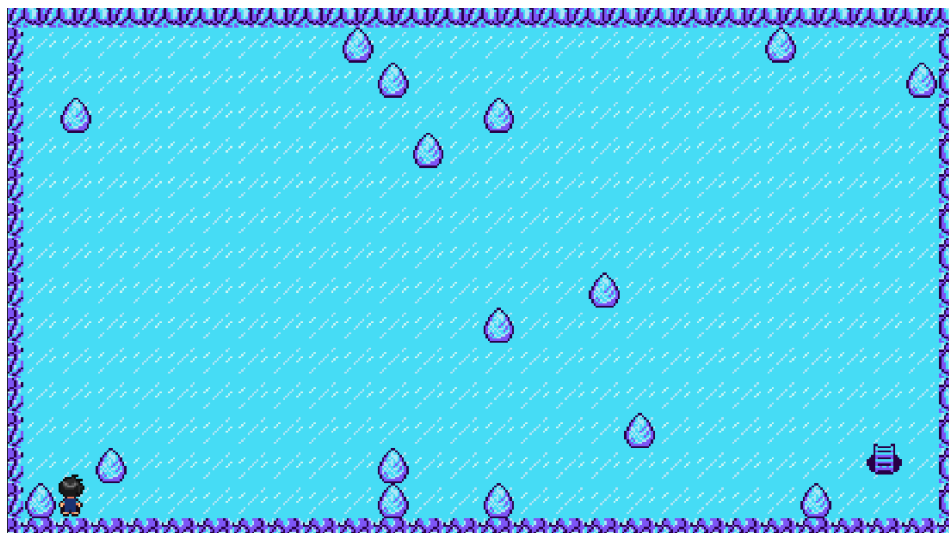


Figura 7. Inicio del nivel 1 del videojuego desarrollado.

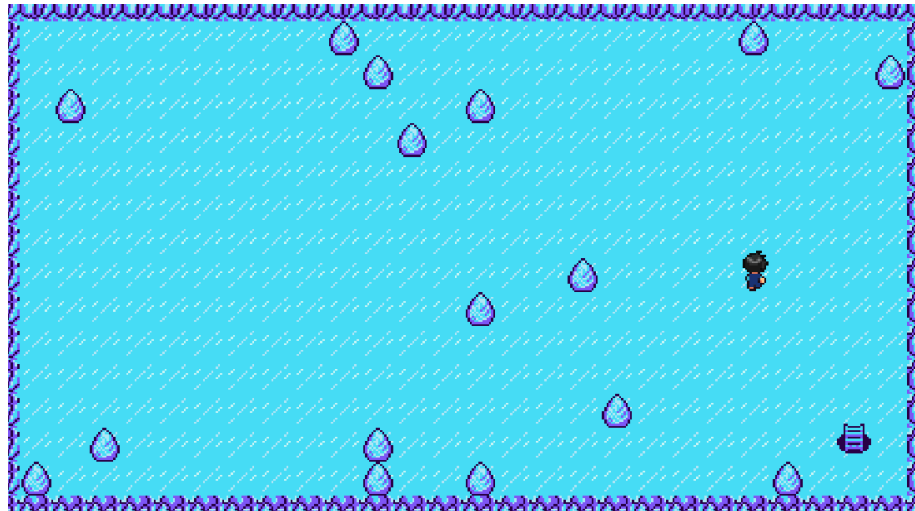


Figura 8. Fin del nivel 1 del videojuego desarrollado.

Implementación

1. CNN simple

La primera red implementada fue la misma que se utilizó durante los trabajos prácticos de la materia “Inteligencia Artificial II”. Esta poseía la siguiente arquitectura:

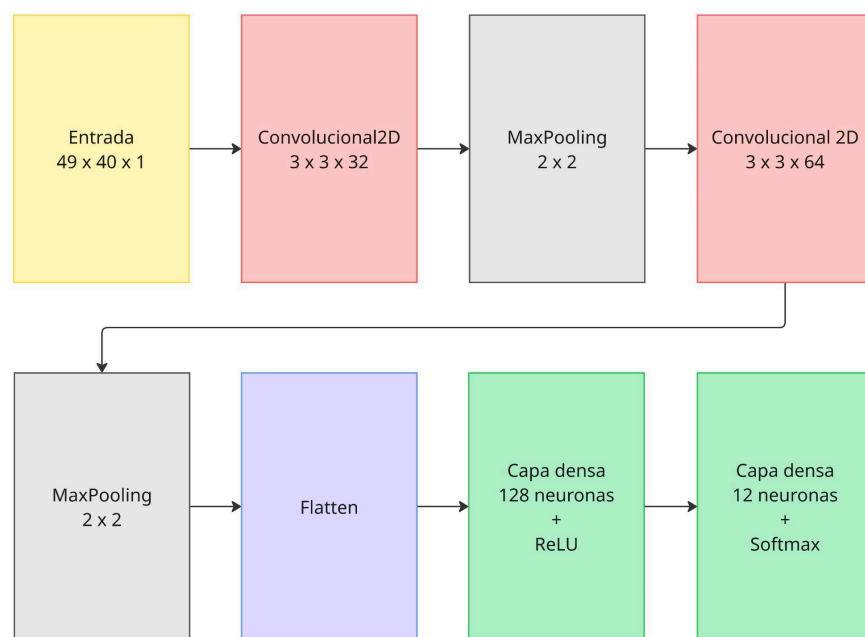


Figura 9. Arquitectura de primera CNN.

En este caso, se hizo undersampling de la clase `_unknown_`, de manera que de esta se mantuvieron solamente 6000 ejemplos. Además, los datos se presentaron con un tamaño de batch de 32. Por último, se utilizaron 15 épocas para entrenar el modelo.

2. CNN con arquitectura VGG16

Se utilizó la arquitectura VGG16 para evaluar su rendimiento en el presente problema. Para ello, se utilizó la implementación de VGG16 disponible en keras, utilizando pesos preentrenados en ImageNet. Sin embargo, la sección del clasificador fue modificada para adecuarse al problema. Además, las entradas se redimensionaron a 224x224 y se convirtieron a tres canales mediante la replicación del canal en escala de grises, con el fin de obtener una dimensión de entrada de 224x224x3. La arquitectura VGG16 final posee la siguiente estructura:

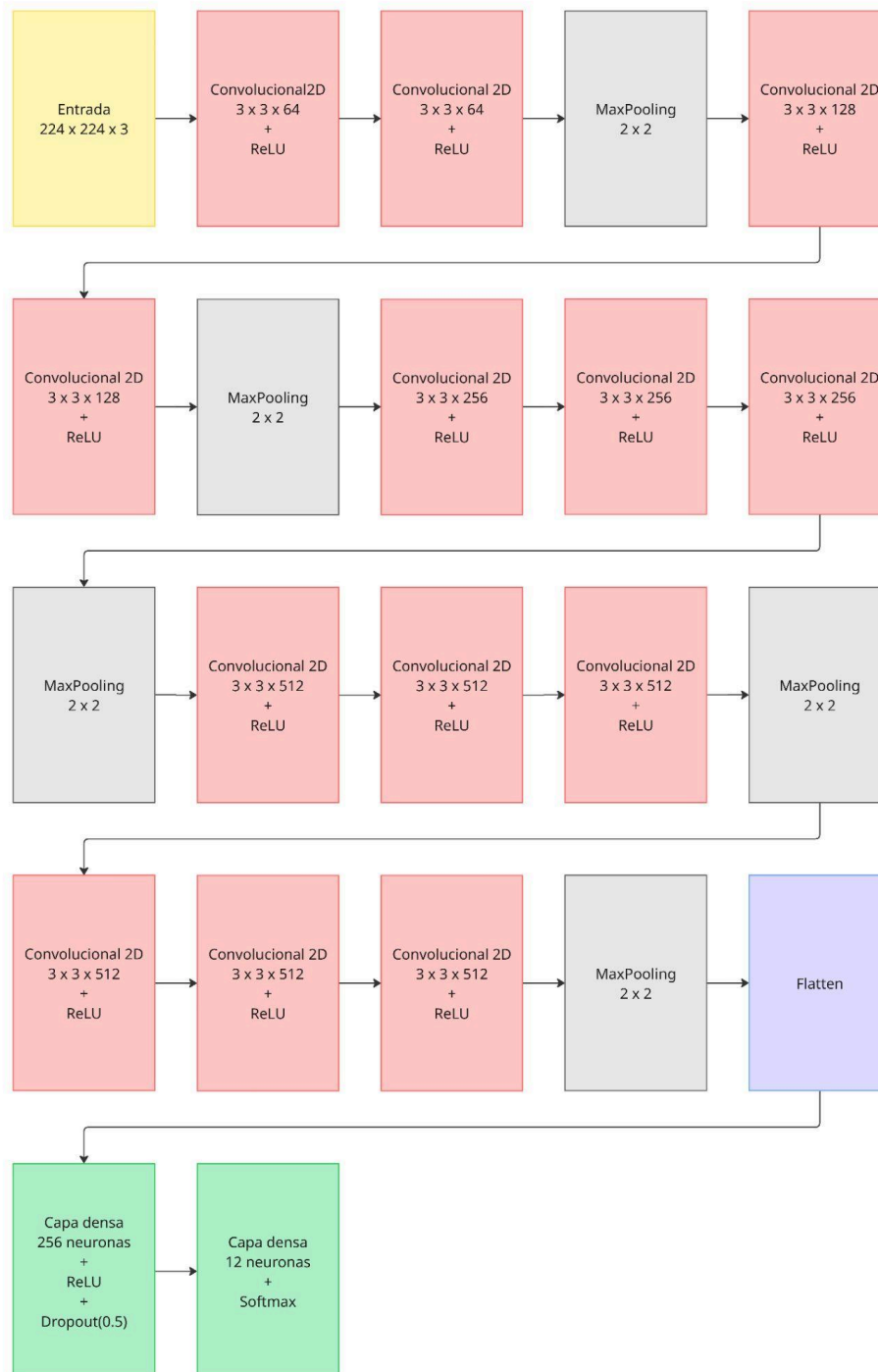


Figura 10. Arquitectura de VGG16.

En este caso se realizó el mismo undersampling que en el caso anterior, es decir, se mantuvieron 6000 casos de la clase `_unknown_`, seleccionados aleatoriamente, el tamaño del batch fue de 32 y también se usaron 15 épocas. En este caso concreto, se intentó agregar diferentes modificaciones para mejorar el resultado final, se agregó dropout y se modificó el clasificador final con diferentes configuraciones.

3. CNN con conexión residual

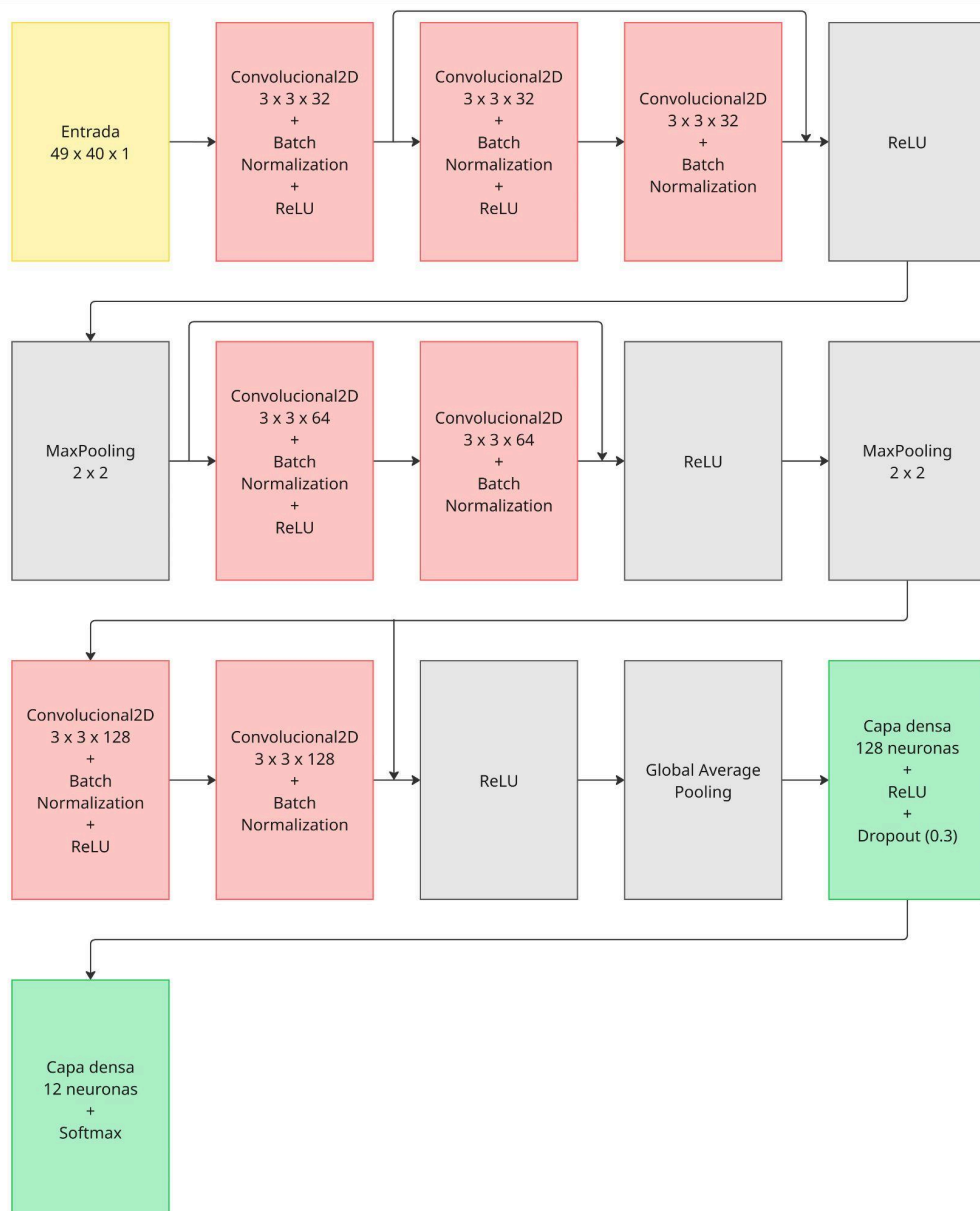


Figura 11. Arquitectura de CNN con conexión residual.

Para este caso, se mantuvieron las mismas configuraciones, es decir, 6000 muestras de `_unknown_`, un tamaño de batch de 32 y 15 épocas. La diferencia con

las anteriores CNN es que en esta se le agregó bloques residuales que implementan conexiones residuales con el objetivo de mejorar el resultado obtenido.

4. Visual Transformers

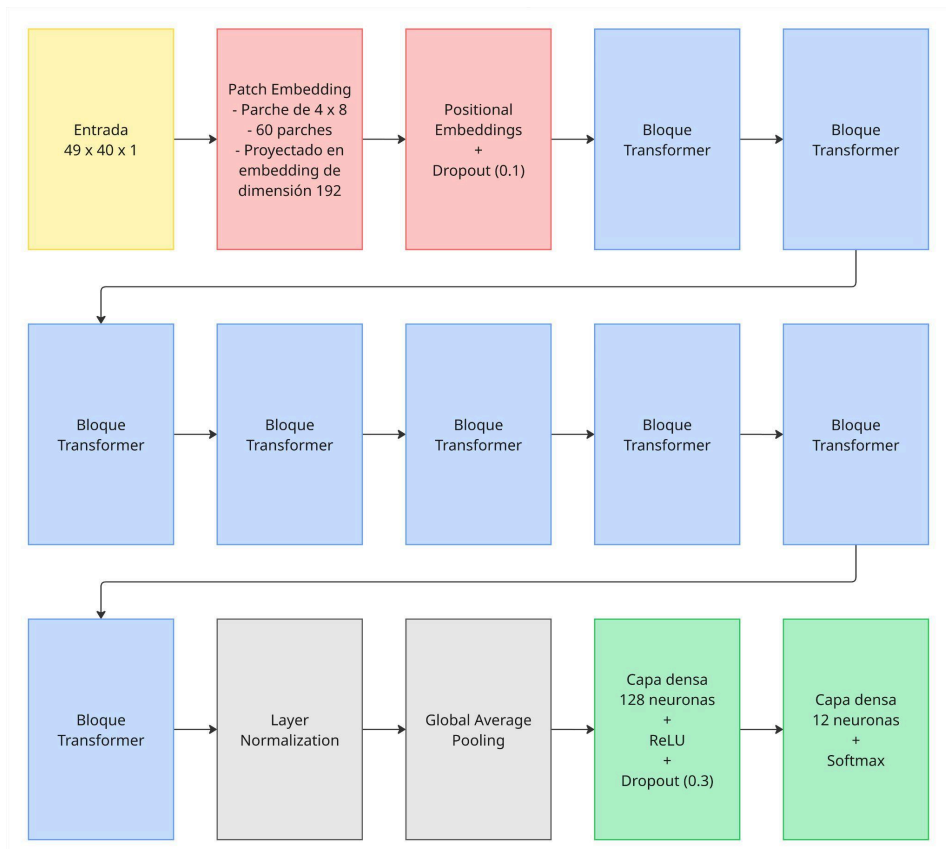


Figura 12. Arquitectura de Vision Transformer.

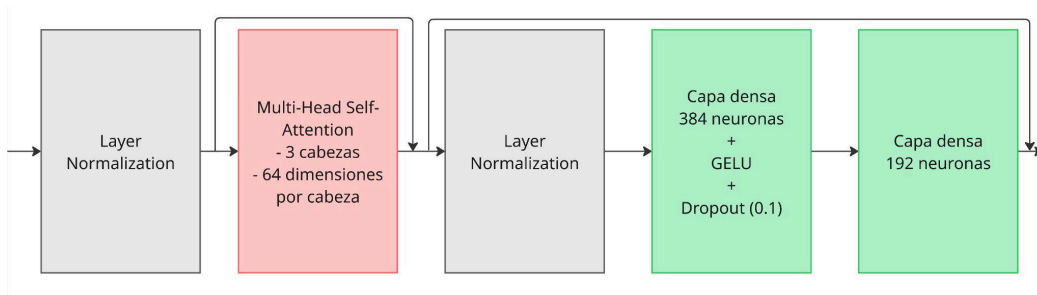


Figura 13. Arquitectura del bloque transformer.

Además, en este caso, se aplicó SpecAugment, la cual es una técnica de data augmentation que actúa sobre los espectrogramas de audio. La misma consiste en enmascarar regiones aleatorias del espectrograma en ambos ejes (tiempo y frecuencia). Con esto se intenta mejorar la capacidad de generalizar del modelo, evitando que no dependa de regiones específicas.

Adicionalmente, en todos los casos se utilizó un tamaño de batch de 32 y 35 épocas, junto a EarlyStopping.

4.1. Visual transformers solo con SpecAugment (v0)

En este caso, no se realizó data augmentation con los audios a excepción de SpecAugment. Es decir, además de lo explicado no se agregó ningún audio adicional, ni se modificaron los existentes para extender el dataset. Además, debido a esto, se mantuvo el mismo undersampling utilizado en todos los casos anteriores, es decir, 6000 casos de la clase `_unknown_`.

4.2. Visual transformers con data augmentation (SpecAugment, ruido, desplazamiento, volumen) (v1)

Para esta instancia sí se realizó data augmentation adicional. Se modificaron los audios originales aplicando diferentes transformaciones. Las mismas fueron:

- Volumen (más alto/ más bajo)
- Desplazamiento (traslación horizontal de la onda)
- Ruido (ruido blanco de fondo)

Las modificaciones se aplicaron con una cierta probabilidad, de manera que por ejemplo, algunos audios se hicieron más fuertes y otros más bajos. Además, se aplicó aleatoriamente la magnitud de la transformación. Por ello, algunos audios son más fuertes que otros, o con mayor ruido, entre otros.

Como resultado, el conjunto de train quedó con una cantidad de 342044 elementos. Ante esto, se aumentó el número de datos de la clase `_unknown_` que se utilizó para entrenar, con un total de 14000.

4.3. Visual transformers con data augmentation (SpecAugment, ruido, desplazamiento, volumen, tono) (v2)

Por último, se volvió a aplicar data augmentation con las mismas modificaciones, pero incorporando una nueva. Las transformaciones finales fueron:

- Volumen (más alto/más bajo)
- Desplazamiento (traslación horizontal de la onda)
- Ruido (ruido blanco de fondo)
- Tono (más grave/más agudo)

Las mismas se aplicaron de la misma manera que en el anterior caso, es decir, aleatoriamente. El conjunto train quedó con una cantidad de 427555 datos y, nuevamente, se elevó la cantidad de elementos del conjunto `_unknown_` a 18000.

Resultados

Tras el entrenamiento, se evaluaron los diferentes modelos obtenidos utilizando las diferentes métricas mencionadas anteriormente sobre el conjunto test. Los resultados se presentan en la siguiente tabla:

Modelo	Accuracy	Precision	Recall	F1 Score	Top-3 Accuracy
CNN	0.8407	0.8850	0.8404	0.8520	0.9569
CNN VGG16	0.7640	0.8173	0.7637	0.7786	0.9319
CNN con conexión residual	0.9515	0.9550	0.9513	0.9523	0.9875
ViT (v0)	0.9278	0.9378	0.9274	0.9301	0.9816
ViT con data augmentation (v1)	0.9321	0.9483	0.9318	0.9360	0.9873
ViT con data augmentation (v2)	0.9491	0.9588	0.9489	0.9513	0.9892

Tabla 1. Resultados de los modelos para el conjunto de test.

Además, se evaluaron en el conjunto de test adicional de 20 elementos con las mismas métricas. Los resultados se presentan a continuación:

Modelo	Accuracy	Precision	Recall	F1 Score	Top-3 Accuracy
CNN	0.7000	0.7407	0.6852	0.6354	0.9500
CNN VGG16	0.3000	0.2879	0.2963	0.2296	0.9000
CNN con conexión residual	0.8000	0.7500	0.7778	0.7074	0.9500
ViT (v0)	0.8000	0.7250	0.7000	0.6800	0.9500
ViT con data augmentation (v1)	0.8500	0.6944	0.7407	0.6889	0.9500
ViT con data augmentation (v2)	0.9000	0.7778	0.8519	0.7926	0.9500

Tabla 2. Resultados de los modelos para el conjunto de test adicional.

Se eligió incorporar al juego el mejor modelo de cada arquitectura. Es decir, el modelo basado en CNN con conexiones residuales y el modelo basado en Visual Transformers con data augmentation v2. Ante esto, se realizaron pruebas manuales para evaluar en la práctica la calidad de los modelos aplicados al videojuego. Al no poder extraer métricas cuantitativas con este tipo de pruebas, se usaron métricas cualitativas con la experiencia de juego.

Para ambos casos, la experiencia de juego fue satisfactoria, comprobándose esto con 3 sujetos de prueba que pudieron completar el nivel varias veces con ambos modelos. En ambos casos se encontró que fallaban al detectar algunos silencios correctamente, pues los detectaban como si fueran instrucciones o “_unknown_”. Analizando esto más a detalle, se observó que una posible causa podría ser la falta de datos conteniendo silencio total en el dataset. Se hicieron pruebas prácticas simulando ruido blanco y se encontró que en la mayoría de los casos lo detectaba correctamente. Igualmente, dicho problema ocurrió en una parte mínima de los intentos, sin llegar a afectar a la experiencia excesivamente, por lo que se optó por mantener el modelo.

Por último, para ambos modelos se realizó la matriz de confusión correspondiente a los dos conjuntos de tests utilizados. Las mismas se presentan a continuación:

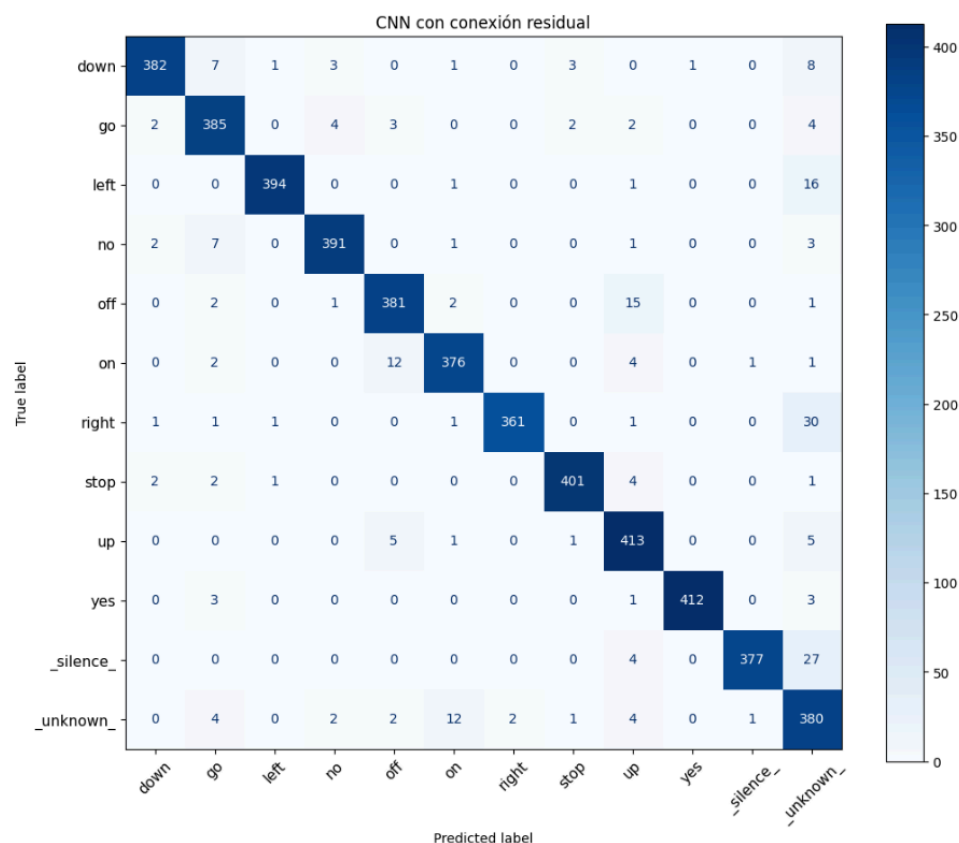


Figura 14. Matriz de confusión de resultados del modelo de CNN con conexión residual para conjunto de test.

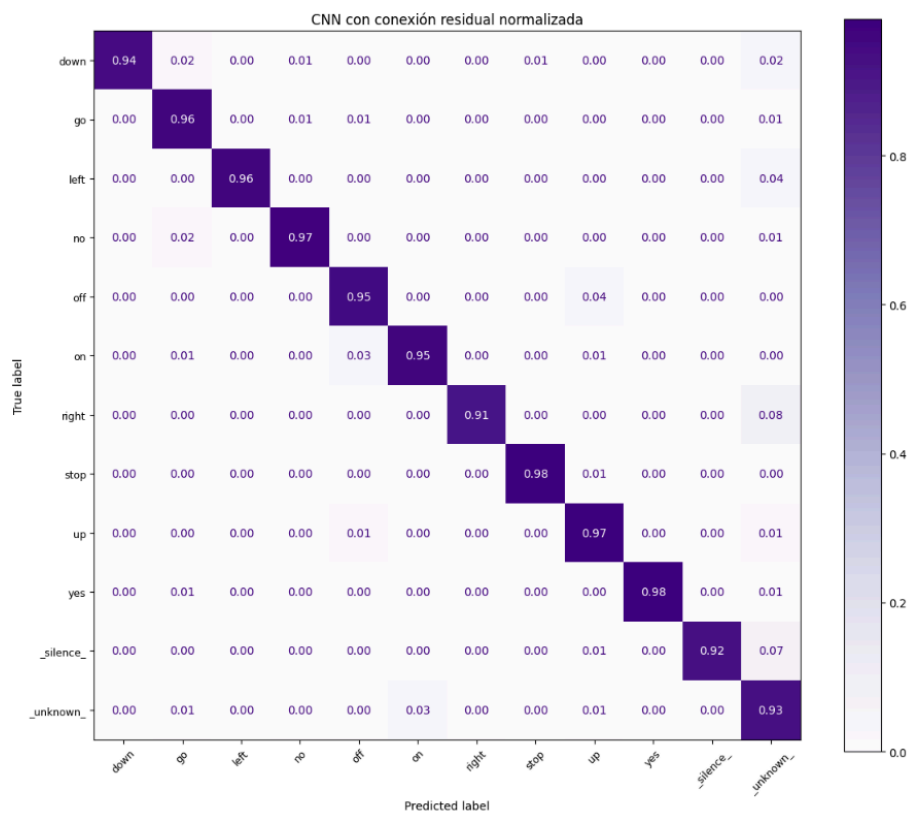


Figura 15. Matriz de confusión normalizada de resultados del modelo de CNN con conexión residual para conjunto de test.

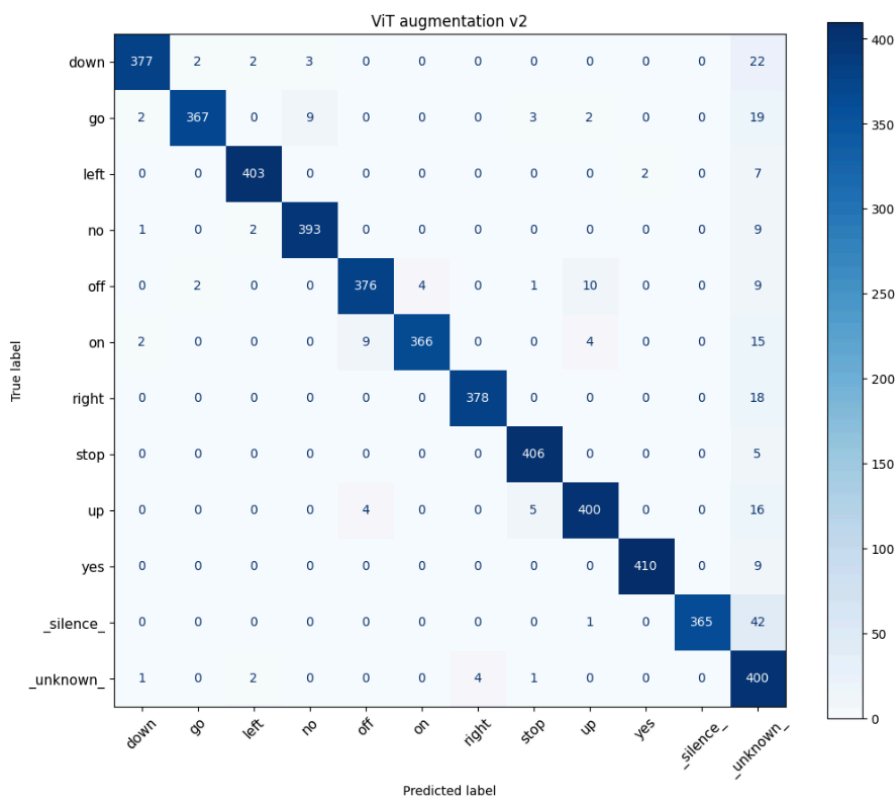


Figura 16. Matriz de confusión de resultados del modelo de ViT v2 para conjunto de test.

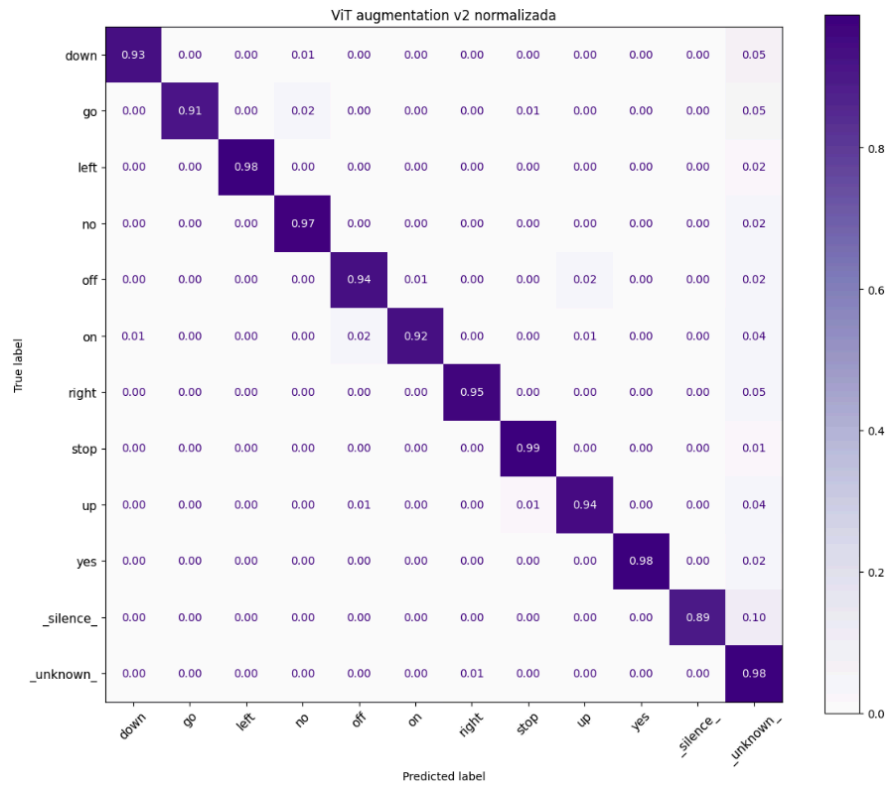


Figura 17. Matriz de confusión normalizada de resultados del modelo de ViT v2 para conjunto de test.

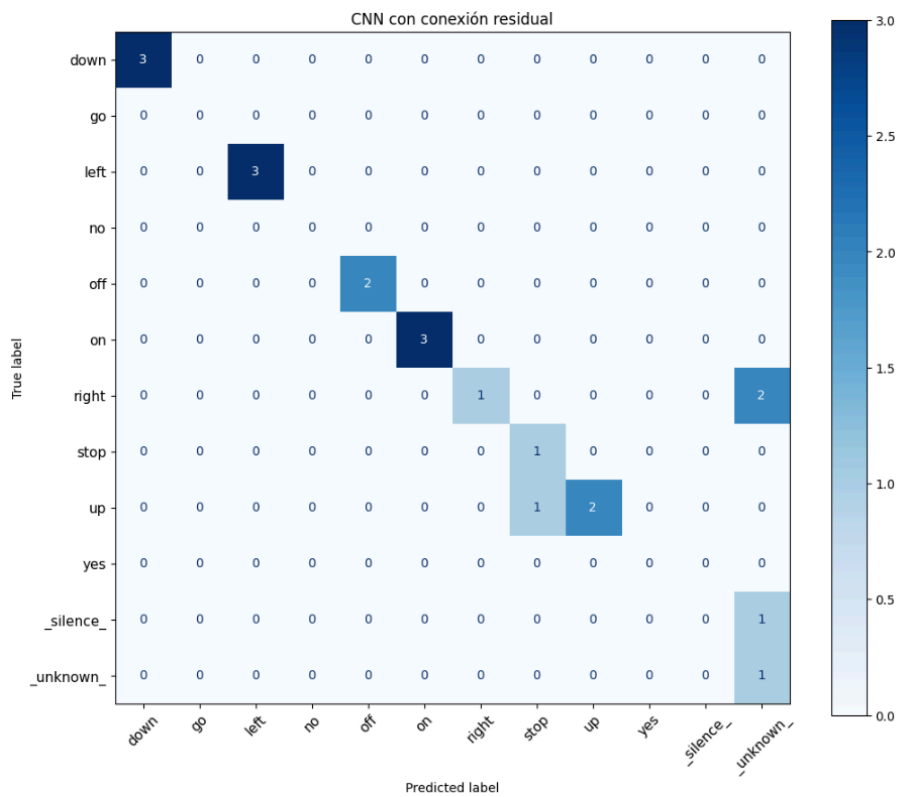


Figura 18. Matriz de confusión de resultados del modelo de CNN con conexión residual para conjunto de test adicional.

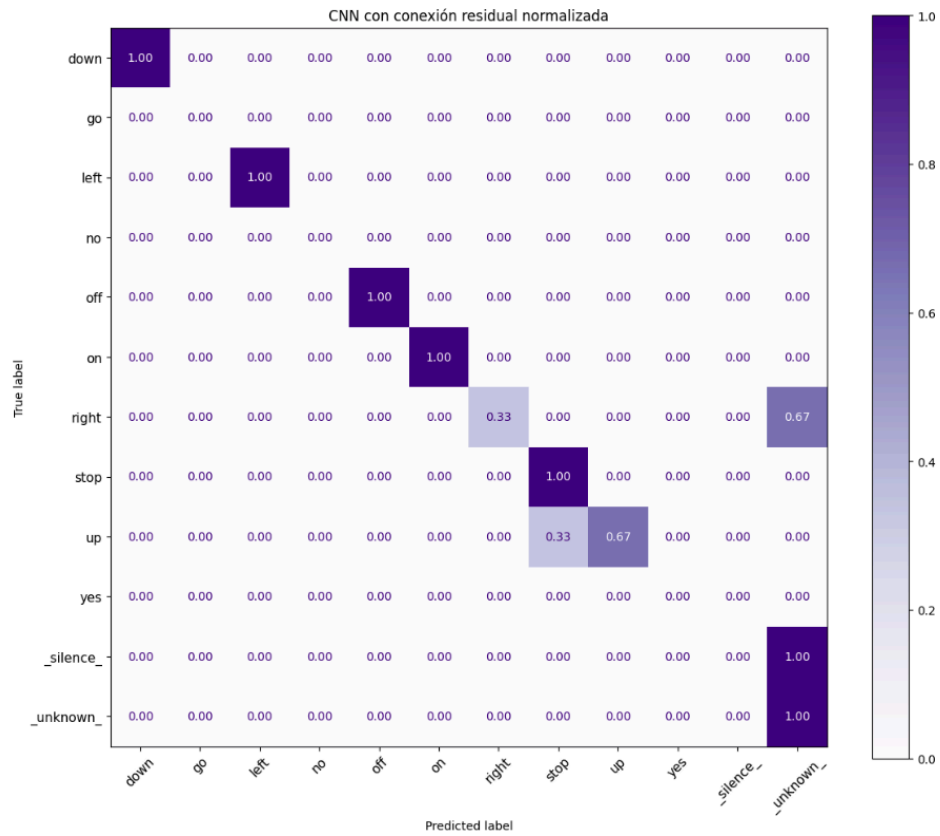


Figura 19. Matriz de confusión normalizada de resultados del modelo de CNN con conexión residual para conjunto de test adicional.

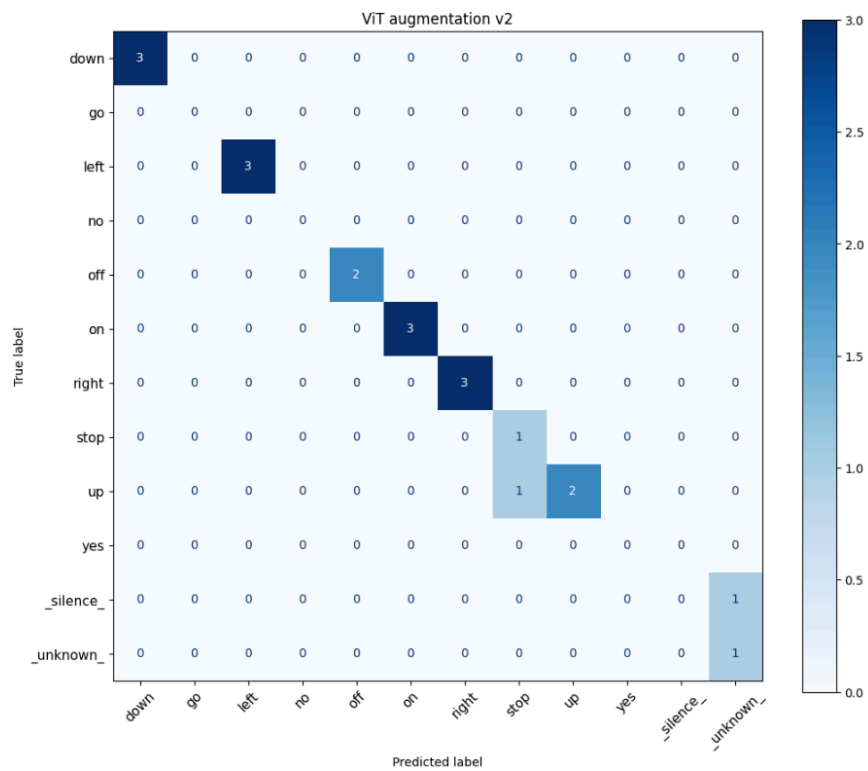


Figura 20. Matriz de confusión de resultados del modelo de ViT v2 para conjunto de test adicional.

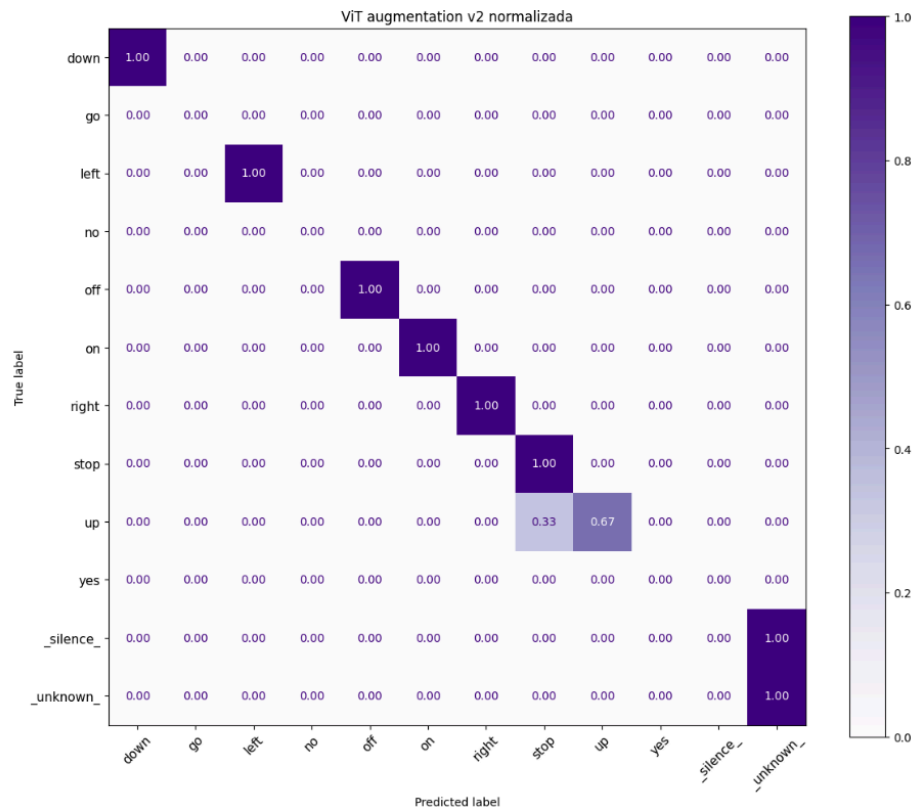


Figura 21. Matriz de confusión normalizada de resultados del modelo de ViT v2 para conjunto de test adicional.

Con respecto a la CNN VGG16, la misma puede no haber rendido tan satisfactoriamente debido a que en la misma se están redimensionando las entradas desde 49x40x1 a 224x224x3. Este procedimiento agrega ruido y puede haber distorsionado ligeramente la estructura del espectrograma, haciendo que la red pueda no haber captado algunos patrones.

Además, otro factor que puede haber influido es que la VGG16 utilizada fue entrenada en ImageNet, el cual no contiene espectrogramas. Por ello, sus filtros pueden no estar bien adaptados para este problema en específico. Se puede concluir que esta arquitectura no es la opción más adecuada para este tipo de problemas y es mejor utilizar otro tipo de soluciones.

Por otro lado, con respecto a los Vision Transformers, específicamente la versión 0 (v0), se experimentó mucho con los hiperparámetros del modelo para obtener los resultados. Se varió la cantidad de cabezas, el tamaño de los embeddings, de los parches, el dropout aplicado, el learning rate, el batch size, la cantidad de épocas, el tamaño del MLP, entre otros. Por esto, la configuración v0 fue la utilizada como punto de partida para mejorar el desempeño del modelo en las siguientes versiones.

En la versión v1 y v2 de la arquitectura con Vision Transformers, es decir, aplicando data augmentation adicional, se encontró que, aunque las mejoras en las métricas fueran moderadas, el uso de data augmentation demuestra ser una estrategia útil para mejorar la robustez del modelo, a costa de un mayor tiempo de entrenamiento. En el caso del

videojuego, conviene hacer uso de estos modelos, particularmente del v2, pues a la hora de jugar, los audios no poseen todas las mismas características, e incluso dependen del entorno donde se juegue.

Además, con respecto a la versión 2, se observó la matriz de confusión correspondiente y se pudo identificar que en este caso, el modelo tuvo mayor cantidad de fallos en la clase “_silence_”, al confundirlo con “_unknown_” y menor cantidad de fallos en la clase “right” con respecto al mejor modelo de CNN. Sin embargo, en este caso se observa que los errores que tuvo el modelo están más distribuidos entre las clases y que la mayor parte fue debido a confundirlos con la clase “_unknown_”. Esto podría indicar que una posible mejora, para una iteración futura, podría hacer un mayor undersampling de la clase “_unknown_” o introducir ponderación de clases.

Los mejores modelos de ambas arquitecturas son los realizados con CNN y conexiones residuales, y ViT con data augmentation (v2). Ambos tuvieron unos resultados muy altos y satisfactorios, por lo que se implementaron ambos dentro del videojuego. Igualmente, es importante notar que a pesar de que ambos tengan resultados similares, en la CNN no hizo falta la utilización de data augmentation. Esto demuestra la gran capacidad que tienen las redes convolucionales para analizar y clasificar audios a partir de sus espectrogramas.

Posibles mejoras

En la implementación actual arquitectura de Vision Transformers, actualmente los parches se aplanan en un vector y se pasan por una capa lineal para crear embeddings. Esto se hizo de esta manera debido a que uno de los objetivos del trabajo era comparar el rendimiento entre CNN y Vision Transformers. Por ello, no se quiso incorporar nada de CNN en ViT, incluso si era solamente para feature extraction. Sin embargo, en un futuro, se podría incorporar este mecanismo para intentar mejorar aún más el desempeño del modelo.

Conclusiones

En conclusión, se lograron obtener resultados sumamente satisfactorios con ambas arquitecturas: CNN y Visual Transformers. Las configuraciones con mejor desempeño fueron la CNN con conexiones residuales y el modelo de Visual Transformers con data augmentation (v2). Ambos modelos alcanzaron desempeño destacable y muy similar entre sí, por lo que ambos se implementaron dentro del juego, en contraposición a la idea inicial de implementar solamente el mejor de ellos.

Sin embargo, se realizaron pruebas para analizar la complejidad temporal de ambos modelos y se encontró que el modelo basado en Visual Transformers se demoraba, en promedio, un 77.22% más de tiempo que su opositor en predecir las entradas. En este caso, no es de suma importancia, pues la respuesta sigue siendo dada en un tiempo menor al medio segundo (0.19s en promedio). No obstante, en otras aplicaciones donde el tiempo de respuesta puede ser más crítico, esto puede hacer que se tenga que utilizar el otro modelo.

Con la incorporación de ambos modelos, se realizaron las pruebas prácticas correspondientes, las cuales arrojaron resultados muy positivos. La experiencia de juego resultó satisfactoria y los errores de interpretación ocurrieron con baja frecuencia.

Teniendo en cuenta todos los resultados obtenidos, se puede afirmar que el proyecto fue exitoso, ya que se logró entrenar modelos capaces de identificar correctamente las órdenes de voz y aplicarlas de manera efectiva dentro de un videojuego desarrollado.

Bibliografía

- [1] Chollet, F. (2021). Image classification. En Deep Learning with Python (2nd ed.). Manning Publications. https://deeplearningwithpython.io/chapters/chapter08_image-classification/
- [2] Fant, G. (1940). A scale for the measurement of the psychological magnitude of pitch. The Journal of the Acoustical Society of America. https://pubs.aip.org/asa/jasa/article/8/3_Supplement/208/716666/A-Scale-for-the-Measurement-of-the-Psychological
- [3] Smith, J. O. (n.d.). Spectrograms. Stanford University, Center for Computer Research in Music and Acoustics. <https://ccrma.stanford.edu/~jos/mdft/Spectrograms.html>
- [4] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Dive into Deep Learning. <https://d2l.ai/index.html>
- [5] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Convolutional neural networks. En Dive into Deep Learning. https://d2l.ai/chapter_convolutional-neural-networks/index.html
- [6] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Transformers. En Dive into Deep Learning. https://d2l.ai/chapter_attention-mechanisms-and-transformers/transformer.html
- [7] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Vision transformers. En Dive into Deep Learning. https://d2l.ai/chapter_attention-mechanisms-and-transformers/vision-transformer.html
- [8] Godot Engine. (s. f.). Godot Engine documentation (stable). <https://docs.godotengine.org/en/stable/>
- [9] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Large-scale pretraining with transformers. En Dive into Deep Learning. https://d2l.ai/chapter_attention-mechanisms-and-transformers/large-pretraining-transformers.html
- [10] Jurafsky, D., & Martin, J. H. (2026). Phonetics and speech feature extraction. En Speech and Language Processing. <https://web.stanford.edu/~jurafsky/slp3/14.pdf>